

Kode ini adalah implementasi dari algoritma **Global Histogram Equalization** (Pemerataan Histogram) dalam bahasa C#.

Tujuan Kode:

Algoritma ini bertujuan untuk meningkatkan kontras gambar. Ia bekerja dengan cara menghitung distribusi kecerahan pixel pada gambar asli, lalu "merentangkan" distribusi tersebut agar memenuhi seluruh spektrum warna (0-255). Hasilnya, detail pada area yang terlalu gelap atau terlalu terang akan menjadi lebih terlihat.

Berikut adalah penjelasan detail per bagian dan per baris kodenya:

1. Deklarasi Namespace dan Class

C#

```
namespace MiniPhotoshop.Logic.Histogram
{
    public class GlobalHistogram
    {
        public Bitmap Apply(Bitmap source)
        {
```

- Ini mendefinisikan ruang lingkup kode (namespace) dan nama kelas.
- `public Bitmap Apply(Bitmap source)`: Ini adalah method utama. Menerima gambar input (source) dan akan mengembalikan gambar baru (dst) yang sudah diproses.

2. Persiapan Data (Setup)

C#

```
int w = source.Width;
int h = source.Height;
int numPixels = w * h;
```

- Menyimpan lebar (w) dan tinggi (h) gambar.
- numPixels: Menghitung total jumlah pixel (lebar × tinggi), ini diperlukan nanti sebagai

pembagi dalam rumus normalisasi.

C#

```
Bitmap dst = new Bitmap(w, h, PixelFormat.Format32bppArgb);
```

- Membuat objek gambar kosong baru (dst) dengan ukuran yang sama dengan sumbernya.
- `PixelFormat.Format32bppArgb`: Menentukan format pixel 32-bit (8 bit Alpha, 8 bit Red, 8 bit Green, 8 bit Blue).

3. Mengunci Memori (LockBits)

Bagian ini sangat penting untuk **performa**. Mengakses pixel satu per satu menggunakan `GetPixel()` sangat lambat. Kode ini menggunakan teknik *Direct Memory Access*.

C#

```
BitmapData srcData = source.LockBits(new Rectangle(0, 0, w, h),  
    ImageLockMode.ReadOnly, PixelFormat.Format32bppArgb);  
BitmapData dstData = dst.LockBits(new Rectangle(0, 0, w, h),  
    ImageLockMode.WriteOnly, PixelFormat.Format32bppArgb);
```

- `LockBits`: Mengunci area memori gambar agar bisa diakses langsung sebagai array byte.
- `srcData` dikunci sebagai *ReadOnly* (hanya baca) dan `dstData` sebagai *WriteOnly* (hanya tulis).

C#

```
int bytes = Math.Abs(srcData.Stride) * h;  
byte[] buffer = new byte[bytes];  
byte[] result = new byte[bytes];
```

- `Stride`: Lebar satu baris gambar dalam satuan *byte* (bukan pixel). Kadang ada *padding* di

akhir baris, jadi kita pakai Stride bukan Width * 4.

- buffer: Array byte untuk menampung data gambar asli (Source).
- result: Array byte untuk menampung data gambar hasil (Destination).

C#

```
Marshal.Copy(srcData.Scan0, buffer, 0, bytes);
```

- Marshal.Copy: Menyalin data raw dari pointer memori (srcData.Scan0) ke dalam array C# biasa (buffer). Sekarang kita bisa memanipulasi buffer dengan cepat.

4. Langkah 1: Hitung Histogram

C#

```
// 1. Hitung Histogram (Frekuensi kemunculan warna)
int[] histR = new int[256];
int[] histG = new int[256];
int[] histB = new int[256];
```

- Menyiapkan 3 array berukuran 256. Setiap index (0-255) akan menyimpan jumlah pixel yang memiliki nilai warna tersebut.

C#

```
for (int i = 0; i < bytes; i += 4)
{
    histB[buffer[i]]++;
    histG[buffer[i + 1]]++;
    histR[buffer[i + 2]]++;
}
```

- Looping menelusuri seluruh byte gambar dengan lompatan 4 (i += 4) karena formatnya

- 32bpp (4 byte per pixel).
- Dalam memori Windows (Little Endian), urutan byte biasanya adalah **B-G-R-A**:
 - buffer[i]: Blue
 - buffer[i+1]: Green
 - buffer[i+2]: Red
 - Kode ini menghitung berapa kali sebuah nilai warna muncul (frekuensi). Contoh: Jika buffer[i] bernilai 100, maka histB[100] ditambah 1.
-

5. Langkah 2: Hitung CDF (Cumulative Distribution Function)

C#

```
// 2. Hitung CDF (Cumulative Distribution Function)
```

```
int[] mapR = CalculateCDF(histR, numPixels);
```

```
int[] mapG = CalculateCDF(histG, numPixels);
```

```
int[] mapB = CalculateCDF(histB, numPixels);
```

- Memanggil fungsi helper CalculateCDF (dijelaskan di bawah) untuk setiap channel warna.
 - Hasilnya adalah array map yang berisi "rumus konversi":
 - Index array = Nilai warna lama.
 - Isi array = Nilai warna baru setelah ditala (equalized).
-

6. Langkah 3: Mapping Pixel (Penerapan Nilai Baru)

C#

```
// 3. Mapping Pixel Lama ke Nilai Baru
```

```
Parallel.For(0, bytes / 4, i =>
```

```
{
```

```
    int k = i * 4;
```

```
    result[k] = (byte)mapB[buffer[k]];    // Map Blue
```

```
    result[k + 1] = (byte)mapG[buffer[k + 1]]; // Map Green
```

```
    result[k + 2] = (byte)mapR[buffer[k + 2]]; // Map Red
```

```
    result[k + 3] = buffer[k + 3];        // Alpha (tidak berubah)
```

```
});
```

- Parallel.For: Menggunakan multi-threading (menggunakan semua core CPU) untuk mempercepat proses. Ini jauh lebih cepat daripada for biasa untuk gambar besar.
- i: Mewakili index pixel ke-sekian.
- $k = i * 4$: Mengkonversi index pixel ke index byte awal.
- Logika Mapping: Kode melihat nilai asli di buffer, menggunakannya sebagai index untuk melihat tabel map, lalu menyimpan nilai baru ke result.
 - Contoh: Jika pixel Biru asli nilainya 50, dan mapB[50] isinya 80, maka pixel baru menjadi 80.
- Alpha (k+3) disalin langsung tanpa diubah agar transparansi gambar tetap terjaga.

7. Menyimpan Kembali dan Membersihkan Memori

C#

```
Marshal.Copy(result, 0, dstData.Scan0, bytes);
source.UnlockBits(srcData);
dst.UnlockBits(dstData);
return dst;
}
```

- Marshal.Copy: Kebalikan dari yang di awal. Menyalin array result (yang sudah dimanipulasi) kembali ke pointer memori gambar tujuan (dstData.Scan0).
- UnlockBits: **Sangat Penting**. Melepaskan kunci memori. Jika lupa, aplikasi bisa crash atau memory leak.
- return dst: Mengembalikan objek gambar baru yang sudah kontrasnya diperbaiki.

8. Helper Method: CalculateCDF

Ini adalah otak matematikanya.

C#

```
private int[] CalculateCDF(int[] histogram, int totalPixels)
{
    int[] map = new int[256];
```

```
long sum = 0;
float scale = 255.0f / totalPixels;
```

- map: Array hasil pemetaan.
- sum: Variabel untuk menampung jumlah kumulatif pixel.
- scale: Faktor skala. Rumus Histogram Equalization adalah $(CDF * 255) / TotalPixel$. Kode ini menghitung $255 / TotalPixel$ di awal sebagai scale agar di dalam loop nanti tinggal melakukan perkalian (perkalian lebih cepat daripada pembagian komputer).

C#

```
for (int i = 0; i < 256; i++)
{
    sum += histogram[i];
    int val = (int)(sum * scale);
    if (val > 255) val = 255;
    map[i] = val;
}
return map;
}
```

- Loop dari intensitas 0 sampai 255.
- `sum += histogram[i]`: Menjumlahkan frekuensi pixel saat ini dengan sebelumnya (Kumulatif).
 - Misal: Jika di level 0 ada 10 pixel, dan level 1 ada 5 pixel. Maka CDF di 0 = 10, CDF di 1 = 15.
- `int val = (int)(sum * scale)`: Menormalisasi nilai kumulatif agar kembali ke rentang 0-255. Ini mendistribusikan ulang (meratakan) histogram.
- `if (val > 255) val = 255`: Penjagaan (Clamping) agar nilai tidak pernah melebihi batas byte maksimum.
- `map[i] = val`: Menyimpan nilai baru untuk intensitas i.

Ringkasan Alur Kerja

1. Ambil data raw gambar ke RAM.
2. Hitung berapa banyak pixel merah, hijau, biru di setiap level terang (Histogram).
3. Hitung distribusi kumulatif dan buat tabel pemetaan baru (CDF).
4. Ganti setiap pixel gambar asli dengan nilai dari tabel pemetaan tersebut secara paralel.
5. Simpan kembali jadi gambar.